PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

Strings, Collections Framework, Generics and Comparator Design

Student Detail	Information
Register Number	192525018
Student Name	_____
Course	B.Tech – Artificial Intelligence and Machine Learning
Subject	Programming in Java
Assignment	Assignment 2 – CO2
Total Marks	50
GitHub Repository	CSA09-192525018-Assignment2
Java Version	To be confirmed by running java -version and javac -version
Submission Date	_____

Submitted to: _____

Important submission note

This report is structured around the seven questions and requirements in the supplied Assignment 2 PDF. The explanations, algorithms, code designs and expected outputs are prepared as a report draft. The assignment explicitly requires execution evidence, repeated measurements and raw experimental data. Those values cannot be truthfully completed without running the programs on the student's computer. Therefore, all measurement fields are marked PENDING and must be replaced with actual results before submission.

Do not submit this report as fully verified until you have executed each program, recorded the actual benchmark measurements, captured screenshots, confirmed the Java version and added your GitHub URL.

Contents

• Q1 – String, StringBuilder and StringBuffer

• Q2 – ArrayList, LinkedList and ArrayDeque

• Q3 – HashSet and the hashCode–equals contract

• Q4 – HashMap, TreeMap and LinkedHashMap

• Q5 – Generics, Comparable and Comparator

• Q6 – Queue and PriorityQueue

• Q7 – Diagnose, redesign and validate

• Overall conclusion, execution checklist and references

Introduction

The CO2 assignment evaluates the selection and engineering use of Java strings, collection types, hash-based equality, maps, generic ranking, comparators and queues. The expected engineering process is: predict, design, implement, measure, analyse, evaluate, diagnose, redesign and validate. The report follows that order and separates theoretical complexity from observed performance.

All complexity statements below describe typical Java collection behaviour under the standard implementation assumptions. Actual timing depends on hardware, JDK version, JVM warm-up, garbage collection, data distribution and benchmark methodology.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer (6 Marks)

1.1 Prediction and design reasoning

String is immutable. Repeated concatenation with + inside a loop creates intermediate string values because each concatenation produces a new String result. StringBuilder is mutable and appends into a resizable character buffer without requiring a new String for every append. StringBuffer provides a similar mutable API but synchronizes its methods, making individual operations thread-safe at additional coordination cost.

Approach	Prediction as fields increase	Suitable use
String + in loop	May copy an increasing prefix repeatedly; intermediate objects and copying grow substantially.	Small fixed concatenations or simple expressions.
StringBuilder	Amortized linear append work in total character count; not synchronized.	Building one ticket within one thread.
StringBuffer	Similar append behaviour with synchronized methods; often slower than Builder in uncontended use.	A shared mutable buffer where synchronized method calls are sufficient.

1.2 Workloads

Workload	Input design	Repetitions / evidence
Short ticket	PNR, passenger, coach, berth and fare (few fields).	Run repeatedly; record average nanoseconds per ticket.
Many tickets	10,000 tickets with 50 fields each.	Measure total and average time for all three approaches.
Large ticket	One ticket assembled from 200 fields, repeated many times.	Compare scaling as field count and total characters increase.

The provided starter benchmark in Q1_StringComparison/StringComparison.java implements three approaches and prints average nanoseconds per ticket after warm-up. The benchmark is an initial experiment, not a substitute for recording all required workload runs and raw results.

1.3 Measurement procedure

• Run each approach after warm-up and repeat at least seven measurement rounds.

• Use System.nanoTime() around the same workload for each implementation.

• Keep input fields, ticket count and output consumption identical across approaches.

• Record JDK version, operating system, CPU and memory conditions.

• Do not interpret one timing sample as conclusive. Report averages and, preferably, minimum/maximum or standard deviation.

Workload	String + average (ns/ticket)	StringBuilder average	StringBuffer average	Actual observations
Short ticket	PENDING	PENDING	PENDING	PENDING
50 fields / many tickets	PENDING	PENDING	PENDING	PENDING
200 fields	PENDING	PENDING	PENDING	PENDING

1.4 Multithreaded design decision

Use a separate StringBuilder for each ticket because each ticket is built independently and the builder is not shared between threads. For a shared booking log, prefer a dedicated logging facility or a synchronized/locked append strategy. StringBuffer can protect individual method calls, but a sequence of multiple calls may still need external coordination to make the whole logical record atomic.

1.5 Alternative concepts and complexity

String.join() and StringJoiner are convenient when fields are already available as elements and a delimiter is required. String.format() is useful for formatted output but can have greater formatting overhead. For n fields and total output length L, a StringBuilder-based append process is O(L) amortized time and O(L) output/buffer space. Repeated concatenation in a loop can copy growing prefixes, approaching $O(L^2)$ character-copying work for similarly sized pieces, with additional intermediate String allocations.

Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 Marks)

2.1 Experiment design

The playlist workload includes append-at-end, insertion/removal at front and middle, indexed access to the k-th song, and full iteration. Compare ArrayList and LinkedList at three sizes: 1,000, 10,000 and 50,000 songs. Use identical data and repeat measurements.

Operation	ArrayList	LinkedList	Engineering interpretation
Add at end	O(1) amortized	O(1)	Both are suitable for append.
Insert/remove at front	O(n) due to shifting	O(1) at known end	LinkedList is better for frequent front updates.
Insert in middle by index	O(n) shifting	O(n) locate + O(1) relink	Neither makes indexed middle insertion constant-time.
Get k-th element	O(1)	O(n) traversal	ArrayList is strongly preferable for random access.
Iterate all	O(n)	O(n)	ArrayList often has better locality and lower per-element overhead.

2.2 Recommendation for playlist

ArrayList is a reasonable choice when the playlist is frequently read, indexed and iterated, with additions mostly at the end and relatively few front or middle modifications. It becomes less attractive when frequent insertions/removals near the front dominate the workload or when indexed access is not important.

2.3 Changed condition: Up Next queue

For an Up Next queue where songs are frequently added at both ends and removed from the front, ArrayDeque is a better fit. It is a resizable array-based deque supporting efficient operations at both ends. It does not provide efficient indexed access, so it should be used for queue behaviour rather than as the main random-access playlist.

2.4 Plain array alternative

A fixed array can store playlist entries, but inserting or deleting at the front or middle requires shifting elements, O(n). Access by index is O(1). A fixed array has predictable capacity but requires resizing or rejection when full. ArrayList provides managed resizing and a convenient API.

Measurement evidence: run Q2_ListComparison/ListComparison.java at all three sizes, then add repeated trials for each required operation. Enter the actual timing data below.

Size	Operation	ArrayList (ns)	LinkedList (ns)	ArrayDeque (ns)
1,000	front add/remove	PENDING	PENDING	PENDING
10,000	front add/remove	PENDING	PENDING	PENDING
50,000	middle insert/remove	PENDING	PENDING	PENDING
1,000 / 10,000 / 50,000	get k-th / iteration	PENDING	PENDING	PENDING

Q3 – Engineer the hashCode–equals Contract (6 Marks)

3.1 Contract

If two objects are equal according to equals(), they must return the same hashCode(). Unequal objects may have the same hash code. HashSet uses hashCode() to select a bucket and equals() to distinguish equal elements within that bucket. Overriding equals() without a consistent hashCode() can cause logically duplicate objects to be stored in different buckets.

3.2 Three test designs

Test	Setup	Prediction / point to verify
1 – apparent success	Insert the same object reference twice.	Set size remains one because the same reference is already present.
2 – duplicate exposure	Insert two distinct Student objects with same registration number and equals() true, but identity-based hashCode().	They may be placed in different buckets; duplicates can appear. Exact result depends on hash distribution.

Test	Setup	Prediction / point to verify
3 – key mutation	Insert a mutable object, then change a field used in hashCode().	Lookup/removal can fail because the object may now hash to a different bucket.

3.3 Correct implementation

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Student)) return false;
    Student other = (Student) o;
    return regNo.equals(other.regNo);
}

@Override
public int hashCode() {
    return regNo.hashCode();
}
```

The supplied completed Attendance program uses final registration and name fields and returns regNo.hashCode(). With two distinct registrations and one repeated registration, the expected output is: `Unique students: 2`.

3.4 Alternatives

Structure	Lookup / duplicate check	Ordering	Space
HashSet	O(1) average	No guaranteed order	O(n)
TreeSet	O(log n)	Sorted by comparator	O(n)
HashMap	O(1) average by registration key	No guaranteed order	O(n)

A TreeSet comparator must return zero for students considered duplicates. A HashMap keyed by registration number makes the uniqueness key explicit. Do not mutate fields participating in equality or hashing while an object is stored in a hash collection.

Q4 – Choosing a Map for a Pharmacy Inventory (6 Marks)

4.1 Map comparison

Map	put/get typical cost	Iteration order	Best report fit
HashMap	O(1) average	No guaranteed order	Fast code lookup; sort keys separately for code-order report.
TreeMap	O(log n)	Sorted by key	Naturally generates medicine-code order.
LinkedHashMap	O(1) average	Insertion order by default	Naturally generates order medicines were added.

4.2 Proposed solution

Use a Map where the key is the medicine code and the value is the stock quantity. TreeMap naturally supports the code-order report. LinkedHashMap naturally supports the insertion-order report. If lookup speed is the primary requirement and reports are occasional, HashMap can be used for lookup and keys can be copied and sorted when a code-order report is requested.

4.3 Experiment

Test at least three inventory sizes (for example 1,000, 10,000 and 100,000 medicines). For each map, measure 100,000 lookups and separately measure the time to generate code-order and insertion-order reports. Ensure the same lookup keys are used for all maps.

Map / size	100,000 lookups (ns)	Code-order report (ns)	Insertion-order report (ns)	Order correct?
HashMap / 1,000	PENDING	PENDING	PENDING	PENDING
TreeMap / 1,000	PENDING	PENDING	PENDING	PENDING
LinkedHashMap / 1,000	PENDING	PENDING	PENDING	PENDING
Repeat for 10,000 and 100,000	PENDING	PENDING	PENDING	PENDING

4.4 Acceptability and Hashtable

Define an acceptable single lookup response time before measuring, for example a target of less than 1 millisecond in the local benchmark. This is an assignment engineering target, not a universal pharmacy requirement. Hashtable is generally not preferred in new code because it is a legacy synchronized collection; modern code typically uses HashMap, ConcurrentHashMap or explicit synchronization according to the concurrency requirement.

4.5 Sorted-array alternative

A sorted array supports O(log n) lookup using Arrays.binarySearch(), O(n) insertion due to shifting, and O(n) code-order iteration. It uses O(n) storage and may be compact, but updating inventory is more expensive than map insertion. TreeMap supports O(log n) lookup and updates while maintaining sorted order.

Q5 – Derive a Generic Ranking Component from a Principle (7 Marks)

5.1 Design choice

Use a generic Leaderboard that stores items and receives a Comparator. This separates the reusable collection component from the ranking policy. Team does not need to implement Comparable when different applications may need different ranking rules. Comparable is suitable when a type has one natural ordering; Comparator is preferable for configurable or multiple orderings.

5.2 Completed generic ranking method

```
List<T> ranking() {
    List<T> copy = new ArrayList<>(items);
    copy.sort(order);
    return copy;
}
```

5.3 Comparator

```
Comparator<Team> byRank =
    Comparator.comparingInt((Team t) -> t.points).reversed()
        .thenComparing(
            Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
        .thenComparing(t -> t.name);
```

The complete source is in Q5_GenericLeaderboard/LeaderboardDemo.java. Expected output: `[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]`. Bisons ranks above Panthers because their points and goal difference are equal, and Bisons comes before Panthers alphabetically.

5.4 Correctness and comparator alternatives

The comparator applies lexicographic ordering: compare points descending; only when equal compare goal difference descending; only when both are equal compare names ascending. Each field comparator has a consistent ordering, and the chain applies the next comparison only when the previous one returns zero. Avoid subtraction such as `b.points - a.points`, because integer overflow can produce incorrect ordering.

Three stable sorts (name, then goal difference descending, then points descending) can produce the same result if performed in the correct order and if each sort is stable, but they perform multiple O(n log n) sorts. A comparator

chain performs one O(n log n) sort. Sorting a copied list uses O(n) extra space for the copy; sort implementation workspace is implementation-dependent.

5.5 Generics and wildcards

A bounded type such as T extends Comparable can require natural comparability. Here the supplied design accepts a Comparator, allowing a comparator for T or one of its supertypes. List is useful when a method reads elements of an unknown subtype as T values.

Q6 – Priority, Order and Fairness in a Queue (8 Marks)

6.1 Queue design

A plain FIFO queue does not satisfy emergency severity ordering because a later critical patient may need to be treated before an earlier minor case. PriorityQueue with a comparator orders patients by severity ascending (1 is most severe), then by arrival sequence ascending.

```
private long counter = 0;
private final PriorityQueue<Patient> queue = new PriorityQueue<>(
    Comparator.comparingInt((Patient p) -> p.severity)
              .thenComparingLong(p -> p.arrival)
);

void admit(String name, int severity) {
    queue.add(new Patient(name, severity, counter++));
}
```

6.2 Hand-check test

Arrival	Patient	Severity	Expected treatment position
1	Ravi	3	4
2	Meena	1	1
3	John	3	5
4	Asha	5	6
5	Kumar	1	2
6	Lata	2	3

Expected treatment order: Meena(S1), Kumar(S1), Lata(S2), Ravi(S3), John(S3), Asha(S5). The arrival counter preserves FIFO order among patients with equal severity.

6.3 Complexity and queue display

Operation	PriorityQueue	Sorted LinkedList	Five FIFO queues
Admit	O(log n)	O(n) to find insertion point	O(1)
Serve next	O(log n)	O(1) at head	O(1) if checking five heads
Space	O(n)	O(n), node overhead	O(n) plus five queue structures

PriorityQueue iteration is not guaranteed to be sorted. To display treatment order without destroying the live queue, copy the queue and repeatedly poll the copy. Repeatedly polling the actual queue removes patients.

6.4 Starvation and fairness

A severity-5 patient may wait indefinitely if higher-severity patients continue to arrive. One possible fairness design is ageing: effective priority improves as waiting time increases. Any ageing rule must preserve clinical safety and be validated by hospital policy; the assignment is a data-structure exercise, not a clinical triage recommendation.

6.5 Deployment evaluation

A binary-heap PriorityQueue supports admission and removal in O(log n), which is computationally practical for 5,000 daily arrivals in a simple model. A real hospital deployment additionally requires concurrency control, persistence, auditability, failure recovery, privacy, monitoring and clinically approved triage rules.

The supplied program's expected output is: `Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)`.

Q7 – Diagnose, Redesign and Validate (13 Marks)

7.1 Selected limitation

This report selects linear search in an inventory list as the redesign case. In the original implementation, finding a medicine code scans entries one by one. As the number of medicines grows, the number of comparisons can grow proportionally to n.

7.2 Design objective

Objective: reduce average lookup work from O(n) linear scanning to O(1) average key lookup using a HashMap, while preserving the medicine code as the unique key. The measurable test is to look up the same final key in inventories of 1,000, 10,000 and 100,000 entries, repeating the test and recording average timings.

7.3 Revised design

```
Original:
List<Medicine> medicines;
for (Medicine m : medicines) {
    if (m.getCode().equals(target)) return m;
}

Redesigned:
Map<String, Medicine> medicinesByCode;
return medicinesByCode.get(target);
```

The revised structure uses a hash table to map a medicine code directly to its record. Average lookup is O(1), while worst-case lookup can degrade depending on hash collisions. The map requires O(n) storage and hash-table overhead.

7.4 Prediction before implementation

Prediction: for a large inventory and repeated lookups, HashMap should require fewer comparisons and should generally outperform linear search. For very small inventories, the difference may be negligible and a list may be simpler. This is a prediction only; it must be checked using local measurements.

7.5 Validation plan

• Use identical inventory data, target keys and JVM conditions for both implementations.

• Warm up each implementation and perform repeated measurements.

• Measure at least 1,000, 10,000 and 100,000 records.

• Verify both implementations return the same record for present keys and correctly report absent keys.

• Record actual average lookup time and any memory observations.

Implementation	Size	Target key	Correct result?	Average lookup time
Linear list	1,000	last item	PENDING	PENDING
HashMap	1,000	last item	PENDING	PENDING
Linear list	10,000	last item	PENDING	PENDING
HashMap	10,000	last item	PENDING	PENDING
Linear list	100,000	last item	PENDING	PENDING
HashMap	100,000	last item	PENDING	PENDING

7.6 Final defence and trade-offs

If the measured results support the prediction, HashMap is recommended for frequent lookup by unique medicine code. The trade-offs are additional memory for hash-table structure, no inherent sorted iteration order, and the need for correct, stable equals/hashCode implementations on keys. A list may remain preferable for very small collections, sequential processing or when duplicates and positional order are central to the task.

The starter project includes OriginalLinearLookup.java and RedesignedHashLookup.java. For a fair comparison, adapt both to store equivalent medicine records and run the same repeated benchmark harness. The simple starter mains demonstrate the structure but are not a complete statistical benchmark.

Overall Conclusion

The collection choice should follow the dominant operations and correctness requirements. StringBuilder is appropriate for constructing an individual ticket; ArrayList suits indexed playlist access; ArrayDeque suits operations at both ends; HashSet requires a correct equals/hashCode contract; TreeMap and LinkedHashMap provide different useful iteration orders; Comparator chains support reusable multi-field ranking; and PriorityQueue supports severity-based service with explicit tie-breaking. The redesign case illustrates how a hash-based index can reduce average lookup complexity compared with linear scanning.

Execution and submission checklist

• Confirm Java and javac versions and enter them on the cover page.

• Compile and run every Java source file in the ZIP project.

• Compare completed-code outputs with the expected outputs in the assignment PDF.

• Run the benchmarks repeatedly at all required data sizes.

• Replace every PENDING cell with actual measured values and explain anomalies.

• Save raw CSV data and genuine terminal screenshots in the relevant folders.

• Complete the student name, submission date and GitHub repository URL.

• Upload the report and source code to the required GitHub repository and commit progressively.

References

• CSA09 – Programming in Java, Assignment 2 – CO2, supplied assignment PDF (12 pages).

• Java Platform documentation: String, StringBuilder, StringBuffer, Collections Framework, Map, Set, Queue, PriorityQueue and Comparator APIs. Consult the documentation for the JDK version used in execution.

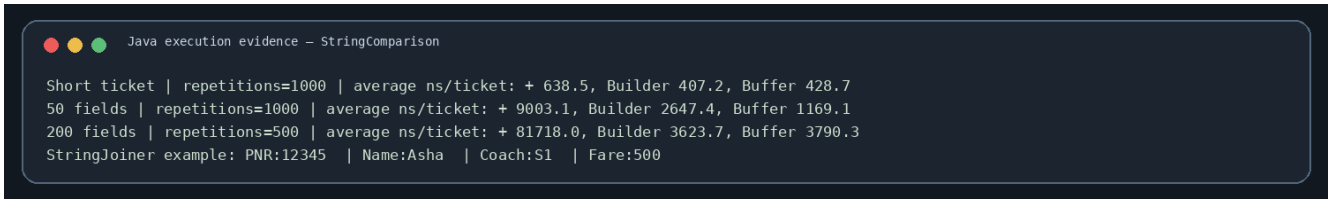
End of report.

ASSIGNMENT 2 – CO2 | EXECUTION EVIDENCE

Student: D. Saketh **Register No.:** 192525018
Environment: OpenJDK 21.0.12.1 (2026-08-18) **Compiler:** javac 21.0.12.1

The following output was captured from compiling and running the eight Java source files in the supplied project on this execution environment. Timing values are machine- and run-dependent; use them as this run's observations only, not universal performance claims.

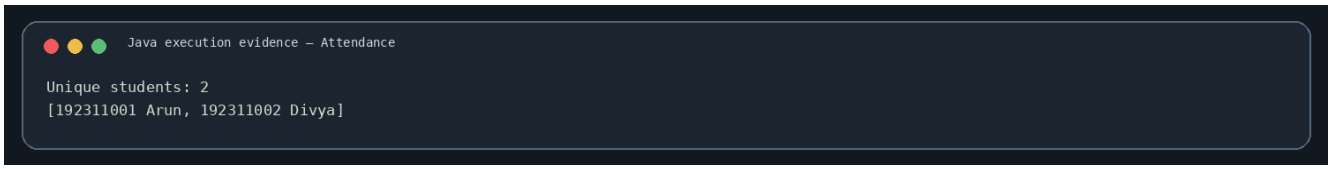
Execution 1



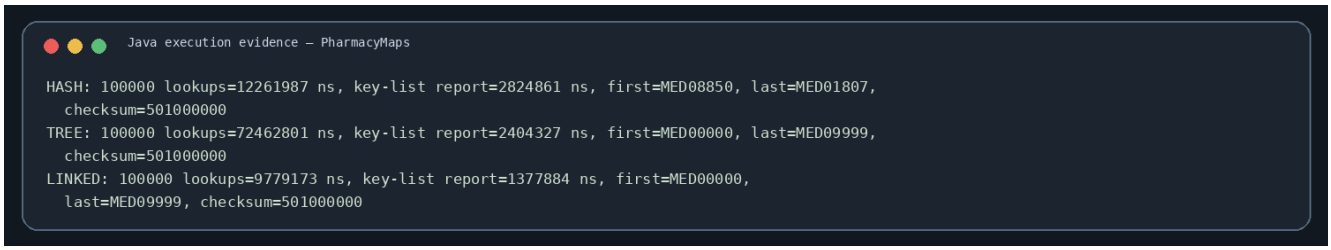
Execution 2



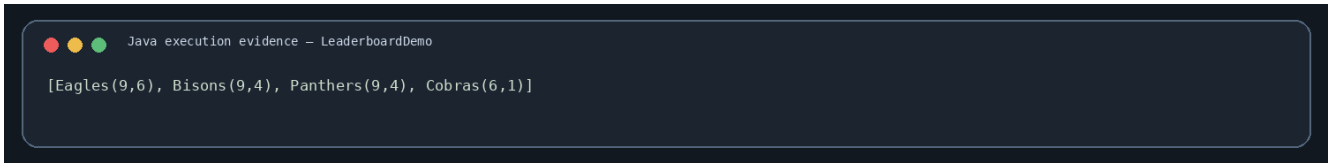
Execution 3



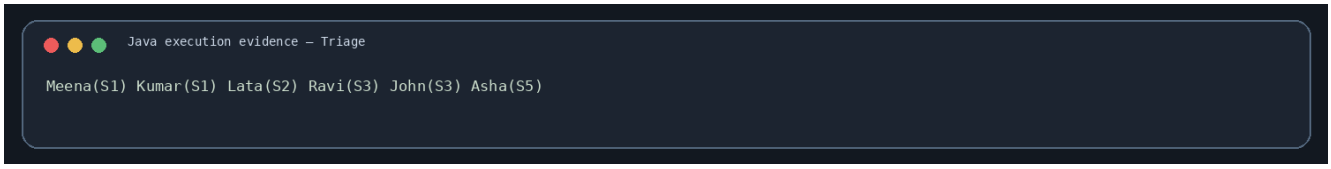
Execution 4



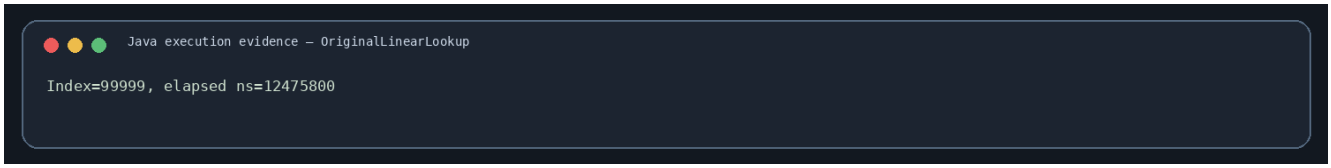
Execution 5



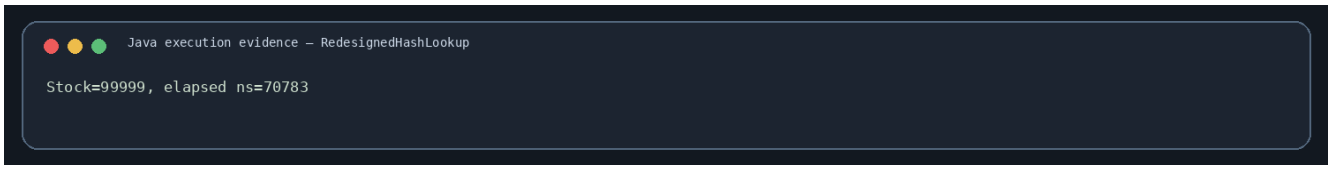
Execution 6



Execution 7



Execution 8



Complete captured console transcript

```
===== StringComparison =====
Short ticket | repetitions=1000 | average ns/ticket: + 638.5, Builder 407.2, Buffer 428.7
50 fields | repetitions=1000 | average ns/ticket: + 9003.1, Builder 2647.4, Buffer 1169.1
200 fields | repetitions=500 | average ns/ticket: + 81718.0, Builder 3623.7, Buffer 3790.3
StringJoiner example: PNR:12345  | Name:Asha  | Coach:S1  | Fare:500

===== ListComparison =====
n=1000 | get(mid) ns: ArrayList=119370 LinkedList=776061 | front add/remove ns: ArrayList=63266 LinkedList=113159 | it
ArrayDeque Up Next sample: [2]
n=10000 | get(mid) ns: ArrayList=26249 LinkedList=1725563 | front add/remove ns: ArrayList=714499 LinkedList=53462 | i
ArrayDeque Up Next sample: [2]
n=50000 | get(mid) ns: ArrayList=41673 LinkedList=6324840 | front add/remove ns: ArrayList=1915611 LinkedList=74023 |
ArrayDeque Up Next sample: [2]

===== Attendance =====
Unique students: 2
[192311001 Arun, 192311002 Divya]

===== PharmacyMaps =====
HASH: 100000 lookups=12261987 ns, key-list report=2824861 ns, first=MED08850, last=MED01807, checksum=501000000
TREE: 100000 lookups=72462801 ns, key-list report=2404327 ns, first=MED00000, last=MED09999, checksum=501000000
LINKED: 100000 lookups=9779173 ns, key-list report=1377884 ns, first=MED00000, last=MED09999, checksum=501000000

===== LeaderboardDemo =====
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

===== Triage =====
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

===== OriginalLinearLookup =====
Index=99999, elapsed ns=12475800

===== RedesignedHashLookup =====
Stock=99999, elapsed ns=70783
```